Finding Metastable Failure Resistant Designs with Bluebell

Zawyar Ur Rehman*
MPI-SWS

Vaastav Anand*
MPI-SWS

Antoine Kaufmann
MPI-SWS

1 Introduction

Metastable failures [5, 11] are a challenging type of emergent mis-behavior [14] of modern cloud systems as they are failure states in which the system exhibits a self-sustaining degradation after a fixed initial trigger. For example, in a retry storm metastability failure [3], a temporary load spike in incoming requests might lead to a cascade of timeouts and retries causing a persistent state of degraded goodput even after the load spike returns to normal. Such failure are costly and have been commonly observed at major cloud companies such as Amazon [18–21], Google [7–9, 17], Meta [4], and others [6, 12, 15, 22].

Due to the high impact and severity of metastable failures, developers must design systems that are less likely to enter a metastable state across a large variety of workloads. However, a system has many design choices such as timeouts, the retry policy, queue/thread-pool sizes, among others, and it is difficult to tell prior to execution which combination will go metastable and when.

To assist developers in analyzing if a specific design will go metastable, recent approaches have proposed to do metastability analysis of a given design of the system using abstract models of the system [1, 10, 13]. For example, Metafor [13] proposes modeling the system as a Continuous Time Markov Chain (CTMC) to perform metastability analysis. The key advantage of such approaches is that they require significantly lower manual effort from developers as developers now only need to provide an abstract representation of the system through the restricted input specifications of the underlying model instead of a full system implementation.

However, these model-based approaches have a fundamental limitation. As these models are abstract representations of the system, they are not entirely complete or sound with respect to the system implementation. Consequently, if a model was to predict the existence of metastable state for a system under specific workload and trigger conditions, it is not guaranteed that the prediction will be accurate.

Consequently, developers have to manually validate predictions on a real deployed system that matches the design to verify that the prediction is correct. As a result, developers must do multiple iterations of the loop where they select a design combination, build a model for that design, use the analysis tools to predict the metastable states, and then manually verify the prediction on the real implementation.

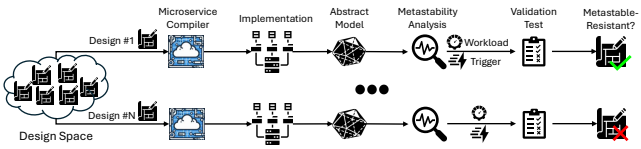

Figure 1: Bluebell Design

To assist developers in finding metastable resistant designs, we are building Bluebell, shown in Figure 1 , an automated design space exploration tool for microservice systems. Bluebell provides a set of abstractions that allow developers to specify the design space they would like to explore. For each candidate design, Bluebell generates a corresponding implementation using the Blueprint microservice compiler [2] and then executes the system to collect performance metrics. Next, Bluebell automatically translates the design parameters and collected metrics into the input specification of an abstract model capable of performing of metastability analysis, such as Metafor [13]. Bluebell uses the analytical capabilities of each model to iterate through a range of workload and trigger combinations to identify the workload-trigger combinations that are predicted to drive the system into a metastable state. Finally, Bluebell certifies these predictions by replaying the corresponding workload and trigger combinations on the generated implementation to determine whether they indeed induce metastability.

By combining implementation-based measurement with model-based analysis, Bluebell enables efficient exploration of the metastability design space for microservice systems.

2 Bluebell Design Sketch

System Design Space. Bluebell uses Blueprint's abstractions of separating the system design space into two specifications: (i) a *workflow specification*, which captures the application's business logic, and (ii) a *wiring specification*, which captures the system's deployment and orchestration choices, including retry policies, timeout configurations, queue sizes, thread pool sizes, and other runtime parameters. Developers implement the workflow specification once, thereby fixing the application's execution DAG at the API level. They then define the design space by specifying the set of candidate values for the configurable features in the wiring specification.

*Student Authors

The resulting design space is the Cartesian product of these configuration choices, with each point corresponding to a distinct system design. For each candidate design, Bluebell combines the workflow specification with the corresponding wiring specification and invokes the Blueprint compiler to generate a concrete microservice implementation.

Extracting Performance Metrics. Bluebell executes an open-loop workload that drives the system to saturation in order to characterize its steady-state performance. During execution, Bluebell collects fine-grained performance metrics from each microservice and computes the effective processing rate of every API in the execution DAG. These measured service rates capture the behavior of the system and are subsequently used as inputs to the abstract model.

Abstract Model Interface. Bluebell is model-agnostic, allowing it to integrate with any abstract model capable of performing metastability analysis. This decouples Bluebell from any particular modeling framework, making it straightforward to incorporate more expressive or accurate abstract models as they become available. To support this flexibility, Bluebell defines an adaptor interface that translates the implementation into the input specification required by the target model. The adaptor consumes the system's performance metrics, design parameters, and execution DAG, and produces the corresponding model specification. Consequently, the adaptor controls the fidelity of the system preserved in the generated model specification. For example, an adaptor may choose to combine multiple APIs for a server into a single API or it may preserve the specifics of the retry mechanism (e.g. with exponential backoff between retries) depending on the limitations of the underlying abstract model.

Metastability Analysis with Abstract Models. To carry out metastability analysis, Bluebell configures the generated abstract model to iteratively try out different workload-trigger combinations from a developer-defined space. Each workload-trigger combination is composed of a stable workload request rate, a trigger request rate, a trigger duration, and the trigger start time. Bluebell performs metastability analysis for each workload-trigger combination and based on the analysis results decides if the particular combination induces a metastable state. If the combination induces a metastable state, then the model produces the combination as the prediction of metastability.

Validating Predictions. Bluebell uses the metastability analysis predictions by replaying the predicted workload-trigger combination on the actual system implementation to validate if the system enters a metastable state. If the system enters a metastable state, then the workload-trigger combination is marked as a valid metastability-inducing combination and the exploration process exits for this specific design. A validated combination acts as a lower bound in the workload-trigger space where the metastability region starts. For any workload that is higher or trigger that is higher or longer, the system is guaranteed to enter a metastable state.

Dealing with False Positives. If the system does not enter a metastable state for a predicted workload-trigger combination, Bluebell treats the combination as a reference point known to be outside the metastable region. It then prunes the search space by discarding all workload-trigger combinations that are no greater than the invalidated point, as metastability can only occur at higher workload and trigger levels. Bluebell subsequently explores the remaining workload-trigger combinations in increasing order until it either discovers a metastability-inducing combination or reaches the configured upper bounds on the workload and trigger dimensions.

Finding False Negatives. False negatives can be identified efficiently only in the vicinity of the predicted metastability boundary. To detect them, Bluebell replays the highest workload-trigger combination that the analysis predicts will *not* induce metastability on the deployed system. If the system remains stable, the prediction is validated. Otherwise, if the system enters a metastable state, Bluebell classifies the workload-trigger combination as a false negative and updates the metastability boundary accordingly.

Preliminary Results. We are currently in the process of implementing Bluebell with different abstract models [1, 10, 13]. We have already integrated Bluebell with Metafor's Koopman Autoencoder model [16]. For our experiment, we design a simple two-service application with Blueprint where each service providing a single API. The API of the first server calls the API of th second server, and each API generates an array of 1000 elements where each element is 1KB in size. We configure the system with one retry on failure, a one second timeout, an initial stable period of 60s and a post-trigger observation period of 60s. We explore 5 different baseline request rates with 3 different triggers for 15 different workload-trigger combinations. We configure Bluebell to explore with Metafor's Koopman-Autoencoder model. While the exploration does not yield any true or false positives, Bluebell's validation phase captures a false negative case as exploration with Metafor predicts no metastable failure for all combinations, the deployed system enters a metastable state with a trigger request rate of 2× the system capacity lasting for 30s. This is because in the deployed system, continuous allocations trigger the garbage collection process which causes resource contention with the resource processing thread. This is something that Metafor's models do not incorporate leading them to fail at finding a metastable state. Analysing one workload-trigger combination with Metafor takes 61s on average. If we configure Bluebell to only use Metafor's Discrete Event Simulator, analysing a combination only takes 4.2s on average. Validating one combination requires $120 - 150s$ depending on the trigger duration.

References

[1] Peter Alvaro, Rebecca Isaacs, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Formal analysis of metastable failures in software systems. *arXiv preprint arXiv:2510.03551*, 2025.

[2] Vaastav Anand, Deepak Garg, Antoine Kaufmann, and Jonathan Mace. Blueprint: A toolchain for highly-reconfigurable microservice applications. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 482–497, 2023.

[3] Microsoft Azure. Retry storm antipattern. Accessed July 2026 from https://learn.microsoft.com/en-us/azure/architecture/antipatterns/retry-storm/.

[4] Nathan Bronson. Solving the mystery of link imbalance: A metastable failure state at scale. Accessed July 2026 from https://engineering.fb.com/2014/11/14/production-engineering/solving-the-mystery-of-link-imbalance-a-metastable-failure-state-at-scale/, 2014.

[5] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, pages 221–227, 2021.

[6] Circle CI. Db performance issue: Incident report for circleci. Accessed July 2026 from https://circleci.statuspage.io/incidents/hr0mm9xmm3x6, 2015.

[7] Google Cloud. Google app engine incident #19007. Accessed July 2026 from https://status.cloud.google.com/incident/appengine/19007, 2019.

[8] Google Cloud. Google compute engine incident #19008. Accessed July 2026 from https://status.cloud.google.com/incident/compute/19008, 2019.

[9] Google Cloud. Google cloud infrastructure components incident #20005. Accessed July 2026 from https://status.cloud.google.com/incident/zall/20005, 2020.

[10] Ali Farahbakhsh, Qingjie Lu, Lorenzo Alvisi, Andreas Haeberlen, and Robbert Van Renesse. Characterizing metastable faults and failures. *arXiv preprint arXiv:2606.00942*, 2026.

[11] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 73–90, 2022.

[12] David Poblador i Garcia. Incident management at spotify. Accessed July 2026 from https://engineering.atspotify.com/2013/6/incident-management-at-spotify, 2013.

[13] Rebecca Isaacs, Peter Alvaro, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Analyzing metastable failures. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, pages 172–178, 2025.

[14] Jeffrey C Mogul. Emergent (mis) behavior vs. complex software systems. *ACM SIGOPS Operating Systems Review*, 40(4):293–304, 2006.

[15] Ben Osborne Panagiotis Moustafellos. Elastic cloud incident report: February 4, 2019. Accessed July 2026 from https://www.elastic.co/blog/elastic-cloud-incident-report-feburary-4-2019, 2019.

[16] Nikhil Singh, Alexandra Bugariu, Mahmoud Salamati, and Rupak Majumdar. Metafor: Analyzing metastability in server systems. Accessed July 2026 from https://github.com/mpi-sws-rse/metafor/, 2025.

[17] Google API Infrastructure Team. Google api infrastructure outage incident report. Accessed July 2026 from https://developers.googleblog.com/google-api-infrastructure-outage-incident-report/, 2013.

[18] The AWS Team. Summary of the amazon ec2 and amazon rds service disruption in the us east region. Accessed July 2026 from https://aws.amazon.com/message/65648/, 2011.

[19] The AWS Team. Summary of the amazon simpledb service disruption. Accessed July 2026 from https://aws.amazon.com/message/65649/, 2014.

[20] The AWS Team. Summary of the amazon dynamodb service disruption and related impacts in the us-east region. Accessed July 2026 from https://aws.amazon.com/message/5467D2/, 2015.

[21] The AWS Team. Summary of the aws service event in the northern virginia (us-east-1) region. Accessed July 2026 from https://aws.amazon.com/message/12721/, 2021.

[22] Tom van der Woerdt. Apache cassandra 13810: Overload because of hint pressure + mvs. Accessed July 2026 from https://issues.apache.org/jira/projects/CASSANDRA/issues/CASSANDRA-13810, 2017.